

Mini Game Hub

Project Report

Aryan(22b1205)

Shresth(22b2211)

April 29, 2026

Contents

1	External Libraries	2
2	Design & Architecture	2
2.1	High-Level Structure	2
2.2	Core Components	3
2.3	Key Design Decisions and Rationale	4
3	Features Implemented	4
4	Directory Structure	5
5	Installation & Setup	6
5.1	Requirements	6
5.2	Recommended Setup	6
5.3	Building the Report	6
6	Usage	7
6.1	Running the Project	7
6.2	Interaction Notes	7
6.3	Direct Launch for Quick Testing	8
7	Retrospective	8
7.1	Challenges and How We Addressed Them	8
7.2	What We Would Do Differently With More Time	8
8	References	9

1 External Libraries

Library / Tool	Purpose in the Project
<code>pygame-ce</code>	Builds the graphical interface for the game menu, board rendering, input handling, buttons, and post-game screens.
<code>numpy</code>	Stores all board states as arrays and powers vectorized rule checks such as Tic-Tac-Toe win detection, Connect Four pattern scans, and Othello disc counting.
<code>matplotlib</code>	Generates the post-game charts that summarize top winners and most-played games. The rendered chart image is then embedded in the Pygame results screen.
<code>bash</code>	Used for the authentication entry point and the terminal leaderboard script. This kept the shell scripting portion of the assignment substantial rather than decorative.
<code>awk</code> , <code>sort</code> , <code>tail</code> , <code>wc</code> , <code>mktemp</code>	Used inside <code>leaderboard.sh</code> to parse <code>history.csv</code> , aggregate per-game statistics, and print a formatted table.
<code>sha256sum</code> / <code>shasum -a 256</code>	Used in <code>main.sh</code> to hash passwords before storing them in <code>users.tsv</code> . The script auto-detects which command exists on the host OS.
<code>tectonic</code>	Used to compile this report into <code>report.pdf</code> through the included Makefile.

Table 1: External libraries and tools used in the project.

In addition to the tools above, the Python standard library plays a supporting role. For example, `pathlib` simplifies cross-platform paths, `subprocess` invokes the Bash leaderboard from Python, `datetime` timestamps recorded matches, and `abc/typing` help structure the shared board-game base class.

2 Design & Architecture

2.1 High-Level Structure

The project is intentionally split into two layers:

1. **Shell entry and persistence layer.** `main.sh` authenticates two players, registers new users when needed, and launches the Python application with the chosen usernames. `leaderboard.sh` reads persistent history from `history.csv` and prints a sortable terminal leaderboard.
2. **Python game engine layer.** `game.py` owns the graphical menu, game selection, result recording, chart generation, and post-game flow. The individual games live under `games/` and inherit shared behavior from `games/base_game.py`.

This split was a deliberate design choice. Authentication and leaderboard reporting map naturally to Bash scripts and text files, while the interactive board games themselves are better handled in Python with Pygame and NumPy.

2.2 Core Components

Authentication and launch flow. The program starts in `main.sh`. It ensures `users.tsv` exists, hashes passwords, enforces that Player 1 and Player 2 are different users, and then launches `game.py` with two validated usernames. This gives the rest of the system a clean contract: once Python starts, it can assume the players are already authenticated.

Game hub orchestrator. `game.py` is the main controller. It:

- shows the menu of available games and leaderboard sort options,
- launches the selected game object,
- records results to `history.csv`,
- calls the Bash leaderboard,
- generates a Matplotlib chart image, and
- shows a post-game screen with summary cards and replay/quit options.

The advantage of this central controller is that each individual game class stays focused on its own rules and rendering, while the hub owns cross-cutting concerns such as history, analytics, and session flow.

Shared board-game base class. `games/base_game.py` provides the common state used by all board games:

- player names,
- current turn,
- a NumPy board array,
- reset/turn-switch helpers, and
- a shared one-step undo snapshot mechanism.

Adding undo here was especially useful because the same pattern was needed across Tic-Tac-Toe, Connect Four, and Othello. By storing the last board state and a small amount of extra per-game metadata, the games can each offer one-step undo without duplicating the full snapshot infrastructure.

Game modules. Each game module combines game-specific rules with its own Pygame rendering code:

- `tictactoe.py`: 10x10 board, five-in-a-row win condition, human-vs-human mode, and a single-player engine mode.
- `connect4.py`: 7x7 gravity-based placement, animated drops, and four-in-a-row detection.
- `othello.py`: legal move scanning in eight directions, disc flipping, pass-turn handling, and end-game score comparison.

Persistent data and reporting. Two text files provide simple persistence:

- `users.tsv` stores usernames and password hashes.
- `history.csv` stores winner, loser, timestamp, and game name for each completed match.

This file-based approach is lightweight, easy to inspect during development, and sufficient for a small local game hub. It also keeps the Bash analytics straightforward because the leaderboard can operate directly on CSV rows.

2.3 Key Design Decisions and Rationale

NumPy boards instead of nested Python lists. Using NumPy for every board gave the project a consistent representation and made the rule checks cleaner. Tic-Tac-Toe and Connect Four particularly benefit because they need repeated pattern detection over a grid. Vectorized operations made those checks concise and less error-prone than manual loops over every possible segment.

Heuristic engine instead of minimax. The single-player Tic-Tac-Toe mode uses a scoring heuristic instead of a full minimax search. That was a practical tradeoff: on a 10x10 board with a five-in-a-row target, a simple heuristic that prioritizes wins, blocks, and central control gives responsive play without introducing expensive search logic or a noticeable UI delay.

Matplotlib rendered to file, then loaded into Pygame. Pygame and Matplotlib do not naturally share the same live UI pipeline. We therefore render charts using Matplotlib's non-interactive Agg backend, save the chart image to `stats.png`, and load that image inside the Pygame post-game screen. This separation avoids event-loop conflicts and keeps chart rendering deterministic.

Leaderboard kept in Bash. We intentionally did not move the leaderboard into Python, even though that would have been possible. Keeping it in Bash preserved the mixed-language nature of the assignment and gave the shell portion a meaningful job beyond startup scripting. The final version also handles game names with spaces correctly, which mattered once `Connect Four` and `Tic-Tac-Toe` both appeared in the same history file.

3 Features Implemented

1. **Two-player authentication and registration.** Players can log in with existing credentials or register new accounts from the terminal. Passwords are stored as SHA-256 hashes rather than plaintext.
2. **Protected two-user session flow.** The script prevents both players from logging in as the same username in the same session.

3. **Graphical game selection hub.** After authentication, the Pygame menu allows users to choose between Tic-Tac-Toe, Tic-Tac-Toe vs Engine, Othello, and Connect Four.
4. **10x10 Tic-Tac-Toe.** Supports human-vs-human play with five-in-a-row win detection across horizontal, vertical, diagonal, and anti-diagonal directions.
5. **Single-player Tic-Tac-Toe engine mode.** One player can challenge the computer. A dedicated selection screen lets either authenticated user choose to play against the engine.
6. **Connect Four.** Implements gravity-based placement, animated coin drops, a hover indicator, and vectorized four-in-a-row detection.
7. **Othello / Reversi.** Implements valid-move hints, disc flipping, skipped turns when a player has no legal move, and end-of-game scoring.
8. **One-step undo in every game.** Each game includes an on-screen Undo button and a keyboard shortcut, restoring the immediately previous move only.
9. **Persistent match history.** Finished games append a row to `history.csv` with winner, loser, timestamp, and game name.
10. **Terminal leaderboard.** The Bash leaderboard prints per-player per-game wins, losses, and win/loss ratios; supports sorting by wins, losses, or ratio; and displays total draw count in the footer.
11. **Readable leaderboard layout.** The leaderboard uses a boxed table, separate game-name and metric rows, and correct handling of multi-word game names.
12. **Post-game analytics.** The results screen shows a chart panel, top-winner summary, most-played-game summary, and total matches played.
13. **Replay flow.** After a finished match, players can either return to the menu and play again or quit the application cleanly.

4 Directory Structure

```

.
|-- .gitignore           - Git ignore rules for generated
    files and local artifacts
|-- Makefile             - Builds report.pdf from report.tex
    using Tectonic
|-- README.md            - Project overview, setup notes,
    and repository documentation
|-- game.py              - Main Python hub: menu, routing,
    stats, history, and post-game UI
|-- games/
|   |-- __init__.py      - Package exports for the game
    classes
|   |-- base_game.py     - Shared board-game base class with
    turn and undo helpers

```

	-- connect4.py	- Connect Four rules, animation,
	rendering, and undo	
	-- othello.py	- Othello rules, valid-move
	scanning, flipping, and undo	
	'-- tictactoe.py	- 10x10 Tic-Tac-Toe, engine mode,
	win detection, and undo	
	-- history.csv	- Persistent match log written
	after completed games	
	-- leaderboard.sh	- Bash leaderboard and CSV
	aggregation script	
	-- main.sh	- Authentication entry point and
	launcher for the Python hub	
	-- mini_game_hub_problem_statement.pdf	
		- Original project brief provided
	for the assignment	
	-- report.tex	- LaTeX source for this report
	-- stats.png	- Generated chart image used by the
	post-game screen	
'	-- users.tsv	- Username/password-hash store for
	registered players	

5 Installation & Setup

5.1 Requirements

- Python 3.10 or newer
- Bash shell
- pygame-ce, numpy, and matplotlib
- Either sha256sum or shasum -a 256

5.2 Recommended Setup

```
python3 -m venv .venv
source .venv/bin/activate
pip install pygame-ce numpy matplotlib
```

No large external assets are required for this project. Everything needed to run the application is already included in the repository, so no Google Drive links or extra downloads are necessary.

5.3 Building the Report

```
make
```

This uses the provided Makefile to compile `report.tex` into `report.pdf`.

6 Usage

6.1 Running the Project

From the repository root:

```
source .venv/bin/activate
bash main.sh
```

The expected flow is:

1. Player 1 logs in or registers.
2. Player 2 logs in or registers with a different username.
3. The graphical menu opens.
4. Players choose a game or inspect leaderboard sort options.
5. After a game ends, the history file is updated, the terminal leaderboard is printed, and the Pygame post-game summary is shown.

6.2 Interaction Notes

- **Tic-Tac-Toe / Othello / Connect Four:** mouse clicks place marks or discs.
- **Undo:** available through the on-screen button and keyboard shortcut.
- **Leaderboard preview:** the menu includes sort buttons that print the terminal leaderboard by wins, losses, or ratio.
- **Single-player mode:** the “Tic-Tac-Toe vs Engine” option asks which authenticated player wants to challenge the computer.
- **Exit flow:** after a finished match, players can either play again or quit from the results screen.

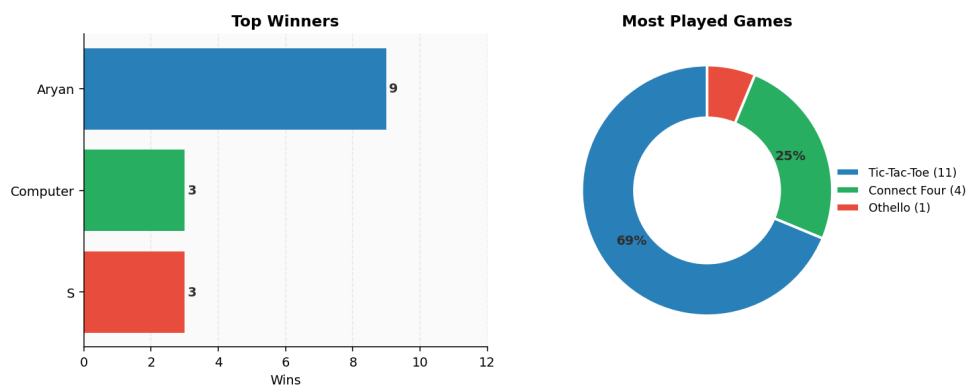


Figure 1: Example statistics chart generated after a completed match.

6.3 Direct Launch for Quick Testing

If the authentication step needs to be bypassed during development, the Python hub can also be launched directly:

```
python3 game.py <username1> <username2>
```

7 Retrospective

7.1 Challenges and How We Addressed Them

Mixed-language coordination. One recurring challenge was keeping the Bash and Python parts aligned. Authentication, result recording, leaderboard parsing, and the Pygame screens all depend on a shared understanding of usernames and history rows. We kept this manageable by standardizing the file formats early: tab-separated credentials in `users.tsv` and comma-separated game history in `history.csv`.

Rule checking without brittle nested logic. The grid-based games each had different rule complexity. Tic-Tac-Toe needed efficient five-cell scans on a large board, Connect Four needed directional checks that were not confused by player-value encoding, and Othello required directional line scanning and pass logic. NumPy helped on the first two, while a clean directional helper approach kept Othello understandable.

Undo across multiple games. Undo sounds simple, but each game also carries small amounts of metadata beyond the board itself. For example, Othello needs to restore skip-delay state and Tic-Tac-Toe needs to clear engine-thinking state. The shared snapshot system in `base_game.py` solved the common part, while each game restored its own additional fields after an undo.

Readable statistics. The first statistics and leaderboard outputs were functional but not very readable. We iterated on both: the post-game chart was made more compact and presentation-friendly, and the terminal leaderboard was reformatted into a clearer boxed layout with a better header structure.

Handling game names with spaces. An unexpected shell-side bug came from using a `for` loop over game names. This split `Connect Four` into two separate tokens. We fixed that by reading unique game names line by line from a temporary file, which made the leaderboard robust for multi-word titles.

7.2 What We Would Do Differently With More Time

- Replace the plain CSV/TSV persistence with a lightweight database or structured JSON model.
- Record draws in a richer format so leaderboard reports can attribute draw counts to individual players instead of only tracking total draws.
- Expand automated testing for core rule logic, especially around Othello edge cases and engine behavior.

- Improve the single-player engine with deeper search or difficulty levels.
- Add richer visuals such as sound, transitions, and custom art assets while preserving responsiveness.

8 References

1. Pygame CE documentation: <https://pyga.me>
2. NumPy documentation: <https://numpy.org/doc/>
3. Matplotlib documentation: <https://matplotlib.org/stable/>
4. Python abc module documentation: <https://docs.python.org/3/library/abc.html>
5. World Othello Federation rules reference: <https://www.worldothello.org>